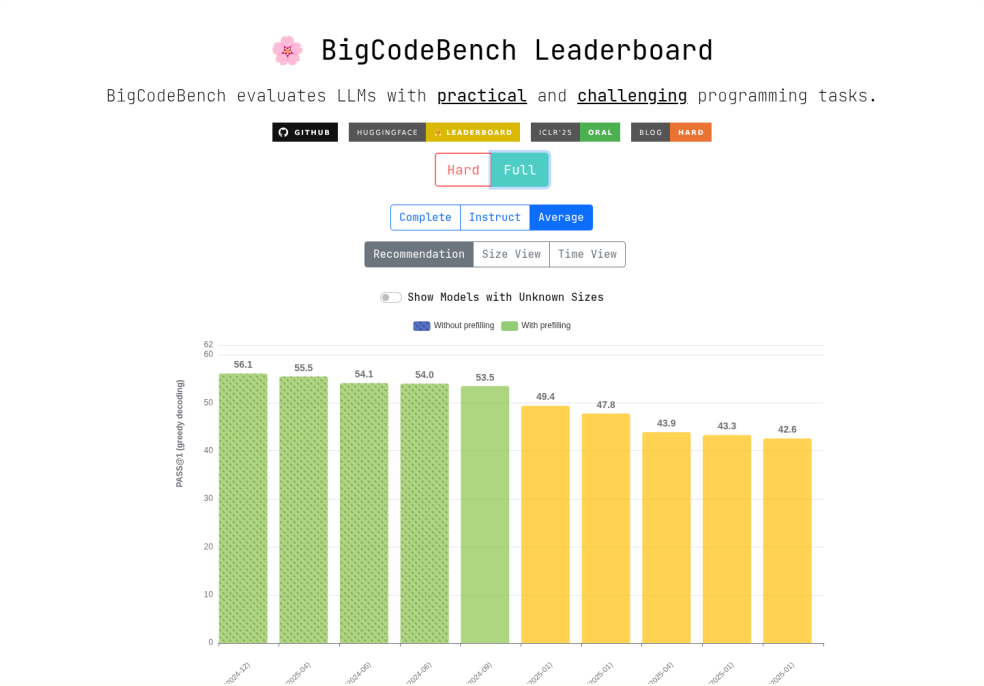
BigCodeBench

複数の Python ライブラリーを組み合わせるコード生成

sasakiy84

BigCodeBench の公式リーダーボード



[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Leaderboard. https://bigcode-bench.github.io/

1. 課題と三つの実例
2. 構築方法と課題の広がり
3. 採点方法とモデルの到達水準
4. 継続利用と入手方法

1. 課題と三つの実例

1,140 課題で、関数を実装する力を測る

- BigCodeBench は、英語で書かれた要件から Python の関数を実装する評価用データセットである。データ分析、ファイル操作、 Web 処理などを扱う。
- 入力は処理の説明と関数の書き出しで、出力は実装するコードである。必要なライブラリーの機能を組み合わせ、指定された戻り値や例外処理を満たす。
- 配布するテストを実行し、要求された動作ができるか採点する。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. https://huggingface.co/datasets/bigcode/bigcodebench

同じ処理を、二つの書き方で依頼する

- Complete は、コード内の文書文字列 docstring で、引数・戻り値・例外・利用例などを詳しく示す。
- Instruct は、自然言語の依頼文とコードの書き出しを示す。同じ機能を、より簡潔な指示から実装する。
- ここでは Instruct の公開 v0.1.4 から、処理の異なる 13 課題を選ぶ。原文には指示とコード指定をすべて示し、日本語では自然言語の指示を訳す。

[1] Zhuo et al. (2025), BigCodeBench. ICLR 2025. https://arxiv.org/abs/2406.15877

例１：辞書を圧縮し、文字列へ変換する

Instruct の原文（全文・コード指定を含む）

Serializes a dictionary to a JSON string, compresses it using zlib, and then encodes the compressed data with base64.
The function should output with:
    str: A base64 encoded string that represents the zlib-compressed JSON string of the dictionary.
You should write self-contained code starting with:

```
import base64
import json
import zlib
def task_func(data_dict):
```

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/25 https://huggingface.co/datasets/bigcode/bigcodebench

例１：日本語訳と期待する動作

日本語訳（自然言語の指示）

辞書を JSON 文字列に変換し、 zlib で圧縮した後、 base64 で符号化する。出力は、圧縮した JSON を表す base64 の文字列である。指定された書き出しを使い、必要なコードをすべて含む実装を書く。

期待する動作：復号・展開すると元の辞書に戻る

配布テストは文字列型を確認したうえで、 base64 を復号し、 zlib を展開して JSON を読み戻す。入れ子のリストや辞書も確かめる。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/25 https://huggingface.co/datasets/bigcode/bigcodebench

例２：指定した数値列を標準化する

Instruct の原文（全文・コード指定を含む）

Standardize specified numeric columns in a dataframe.
The function should raise the exception for: ValueError: If 'df' is not a DataFrame, 'cols' is not a list, or columns in 'cols' don't exist in 'df'.
The function should output with:
    DataFrame: The dataframe with standardized columns.
You should write self-contained code starting with:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
def task_func(df, cols):
```

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/140 https://huggingface.co/datasets/bigcode/bigcodebench

例２：日本語訳と期待する動作

日本語訳（自然言語の指示）

データフレームの指定した数値列を標準化する。 df が DataFrame でない場合、 cols がリストでない場合、指定した列が存在しない場合は ValueError を送出する。標準化した列を含むデータフレームを返す。指定された書き出しを使い、必要なコードをすべて含む実装を書く。

期待する動作：平均は約 0 、標準偏差は約 1

配布テストは A ・ B 列の平均と標準偏差を確認し、列が存在しない場合などの例外も調べる。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/140 https://huggingface.co/datasets/bigcode/bigcodebench

例３： Web ページから最初のタグを読む

Instruct の原文（全文・コード指定を含む）

Scrape a web page for the first occurrence of a specified HTML tag and return its text content.
The function should output with:
    str: The text content of the specified HTML tag if found, otherwise returns None.
You should write self-contained code starting with:

```
import requests
from bs4 import BeautifulSoup
def task_func(url, tag):
```

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/32 https://huggingface.co/datasets/bigcode/bigcodebench

例３：日本語訳と期待する動作

日本語訳（自然言語の指示）

Web ページから、指定した HTML タグの最初の出現を探し、そのテキストを返す。タグが見つからなければ None を返す。指定された書き出しを使い、必要なコードをすべて含む実装を書く。

配布テストの HTML （ title 部分）

```
<title>Test Page</title>
```

期待する動作： title タグの文字列は "Test Page"

タグが存在しない場合に None を返すことも確かめる。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/32 https://huggingface.co/datasets/bigcode/bigcodebench

2. 構築方法と課題の広がり

モデルによる生成を、専門家が繰り返し点検した

- 2024 年に公開され、 ICLR 2025 で発表された。短いアルゴリズムだけでなく、複数のライブラリーを使う実用的な処理を評価するために作られた。
- ODEX のコード例を種に GPT-4 で課題を拡張し、関数名の置換や説明の言い換えを行った。 4,718 件の候補から、コードとテストを整える。
- 人が生成を指導し、テストの不足や実装の不備を修正した。最終的に 1,140 課題を残した。

[1] Zhuo et al. (2025), BigCodeBench. ICLR 2025. https://arxiv.org/abs/2406.15877

20 人の作業で、説明・実装・テストをそろえた

- 構築には著者 20 人が計 1 年間にわたり参加し、 75% は Python の経験が 5 年以上あった。
- 説明と実装の一致、必要なインポート、例外処理、テストの安定性を人が確認した。別の担当者による相互点検も行った。
- 最終確認では 11 人が計 33 課題を解き、 32 課題で全テストを通過した。これは構築時の標本による確認である。

[1] Zhuo et al. (2025), BigCodeBench. ICLR 2025. https://arxiv.org/abs/2406.15877

多くの処理で、複数のライブラリーを使う

項目	値
Python の課題数	1,140
利用するライブラリー数	139
参照実装 1 課題のライブラリー数の平均	2.8
1 課題のテスト数の平均	5.6
参照実装の平均分岐網羅率	99%

分岐網羅率は、参照実装の条件分岐をテストがどれだけ通るかを表す。

[1] Zhuo et al. (2025), BigCodeBench. ICLR 2025. https://arxiv.org/abs/2406.15877

計算・汎用処理・可視化が多い

ライブラリーの分野	使用する課題 (%)
計算	63
汎用処理	44
可視化	31
システム	30

同じ課題を複数の分野に数える。

[1] Zhuo et al. (2025), BigCodeBench. ICLR 2025. https://arxiv.org/abs/2406.15877

時間・ネットワーク・暗号処理も扱う

ライブラリーの分野	使用する課題 (%)
時間	10
ネットワーク	8
暗号・符号化	5

公式の Cryptography には、 base64 などの符号化も含まれる。

[1] Zhuo et al. (2025), BigCodeBench. ICLR 2025. https://arxiv.org/abs/2406.15877

Hard は、実用的で難しい 148 課題を選ぶ

- Full は 1,140 課題全体を指し、 Hard はその中から選んだ 148 課題の部分集合である。どちらにも Complete と Instruct がある。
- Hard は Stack Overflow の利用者の質問と似た課題を取り出し、使うライブラリーの数、参照解の長さ、モデルの解決率などで絞った。
- 限られた計算予算で、難しい課題を評価したい場合に使う。

[1] Zhuo (2024), Introducing BigCodeBench-Hard. https://huggingface.co/blog/terryyz/bigcodebench-hard

3. 採点方法とモデルの到達水準

全テストを通る課題の割合を Pass@1 にする

- 各課題で一つの実装を作り、その課題に付いたテストをすべて通れば成功とする。成功した課題の割合が Pass@1 である。
- 配列の値だけでなく、生成ファイル、例外、グラフの情報、模擬した通信の結果などもテストする。
- 2024 年の主な比較は、解答例を示さない zero-shot と、次のトークンごとに最も確率が高い候補を選ぶ greedy decoding を使った。

[1] Zhuo et al. (2025), BigCodeBench. ICLR 2025. https://arxiv.org/abs/2406.15877

省いたインポートを補う採点も報告する

- 生成コードから必要な部分を取り出した後、そのまま実行する成績と、欠けた書き出しを補う成績を計算する。
- calibrated Pass@1 は、欠けたインポートや定数などを入力のコード指定から補った成績である。
- 公開表には、コードの書き出しを回答の先頭に事前投入する prefilling の設定もある。採点時に欠けた部分を補う calibration と、生成を始める設定を分けて示す。

[1] Zhuo et al. (2025), BigCodeBench. ICLR 2025. https://arxiv.org/abs/2406.15877

初期評価でも、 Instruct の得点が低い

モデル（ 2024 年）	Complete(%)	Instruct(%)
GPT-4o (05-13)	60.2	49.9
Claude 3 Opus	56.8	45.2
DeepSeek Chat V2	49.4	40.4

Full 1,140 課題、 zero-shot ・ greedy 。そのまま実行した Pass@1 。

[1] Zhuo et al. (2025), BigCodeBench. ICLR 2025. https://arxiv.org/abs/2406.15877

公式公開表には、後続モデルの評価も残る

モデル	事前投入	Complete(%)	Instruct(%)
GPT-4.1 Mini (2025)	なし	59.3	48.9
DeepSeek V3 (2024)	なし	62.2	50.0
Qwen2.5 Coder 32B (2024)	あり	58.0	49.0

Full 、 calibrated Pass@1 。 2026-10-22 取得の公開表。

[1] BigCode Project (n.d.; 2026-10-22 参照), official BigCodeBench results. https://huggingface.co/datasets/bigcode/bigcodebench-results

Hard では、同じモデルも得点が下がる

モデル	事前投入	Complete(%)	Instruct(%)
GPT-4.1 Mini (2025)	なし	33.8	31.8
DeepSeek V3 (2024)	なし	40.5	28.4
Qwen2.5 Coder 32B (2024)	あり	33.8	27.7

Hard 148 課題、 calibrated Pass@1 。

[1] BigCode Project (n.d.; 2026-10-22 参照), official BigCodeBench-Hard results. https://huggingface.co/datasets/bigcode/bigcodebench-hard-results

4. 継続利用と入手方法

2026 年には、追加訓練後の能力確認にも使われる

- Lee らは、エージェントに不正な振る舞いを自己申告させる訓練を行い、一般的な能力が保たれるか調べた。
- 七つの評価の一つに、 BigCodeBench の 1,140 課題を採用した。公開コードは Complete 形式を使う。
- GPT-4.1 、 GPT-4.1 mini 、 Gemini 2.0 Flash の訓練前後を比較する。ここではコード生成能力を保てるかを測る役割を持つ。

[1] Lee, Chen and Korbak (2026), Training Agents to Self-Report Misbehavior. https://arxiv.org/abs/2602.22303

公式配布から、二つの提示文とテストを読める

- Hugging Face の bigcode/bigcodebench で、 1,140 課題を申請なしで取得できる。ここで使った v0.1.4 には、依頼文、書き出し、参照実装、テストがそろっている。
- Complete と Instruct の両方を同じ課題 ID で確認できる。v0.1.4 はデータの改訂版を示す。
- 公式 GitHub には評価用の実装と環境の案内があり、 Full と Hard 、提示形式を選べる。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. https://huggingface.co/datasets/bigcode/bigcodebench

1 件の依頼とテストを確認する

```
from datasets import load_dataset

ds = load_dataset("bigcode/bigcodebench",
                  split="v0.1.4")
item = ds[25]
print(item["instruct_prompt"])
print(item["test"])
```

依頼文とテストを同じレコードで読み、求められる動作を確かめる。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. https://huggingface.co/datasets/bigcode/bigcodebench

データと評価コードは Apache 2.0 で配布される

- 公式データカードとコードは Apache License 2.0 を指定する。利用・改変・再配布ができ、再配布にはライセンス文や必要な帰属表示を保つ。
- 実行環境には、対象課題が使う Python ライブラリーも必要になる。公式に用意された評価環境から始めると、依存関係をそろえやすい。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench official repository. https://github.com/bigcode-project/bigcodebench

ライブラリーを組み合わせ、指定の動作を実装する

- BigCodeBench は、実用的な Python 処理を関数単位で実装する 1,140 課題である。 139 ライブラリーにまたがる機能の組合せと、複数の要件への対応を評価する。
- 同じ課題の詳しい Complete と簡潔な Instruct 、全体の Full と難しい Hard を目的に合わせて選ぶ。
- 戻り値だけでなく、ファイルや例外などを確認するテストから、何を実装できたかを具体的に調べられる。

[1] Zhuo et al. (2025), BigCodeBench. ICLR 2025. https://arxiv.org/abs/2406.15877

Appendix

1. 多様な処理を示す追加例
2. 提示形式と利用条件
3. 参考文献

例 4 ：ファイルを ZIP へまとめる

Instruct の原文（全文・コード指定を含む）

Zips all files (not including subdirectories) located in the specified directory and returns the path to the created zip file.
Note that: Notes: The zip name is always 'files.zip'
The function should raise the exception for: FileNotFoundError: if the specified directory does not exist
The function should output with:
    str: The path to the generated zip file. Returns None if the directory does not contain any files.
You should write self-contained code starting with:

```
import os
import glob
import zipfile
def task_func(directory):
```

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/19 https://huggingface.co/datasets/bigcode/bigcodebench

例 4 ：日本語訳と期待する動作

日本語訳（自然言語の指示）

指定したディレクトリのファイルを ZIP にまとめ、生成した ZIP のパスを返す。サブディレクトリは含めない。ファイル名は常に files.zip とする。指定ディレクトリが存在しなければ FileNotFoundError を送出し、ファイルが一つもなければ None を返す。指定された書き出しを使い、必要なコードをすべて含む実装を書く。

期待する動作：直下のファイルだけを ZIP に含める

配布テストは直下とサブディレクトリに各 1 ファイルを置き、 ZIP に直下の 1 ファイルだけが入ることを確認する。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/19 https://huggingface.co/datasets/bigcode/bigcodebench

例 5 ：指定語を除いて単語の頻度を数える

Instruct の原文（全文・コード指定を含む）

Count the frequency of each word in a text after removing specific stopwords.
The function should output with:
    Series: A pandas Series with word frequencies excluding the words in STOPWORDS list.
You should write self-contained code starting with:

```
import pandas as pd
import regex as re
# Constants
STOPWORDS = ["a", "an", "the", "in", "is", "are"]
def task_func(text):
```

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/52 https://huggingface.co/datasets/bigcode/bigcodebench

例 5 ：日本語訳と期待する動作

日本語訳（自然言語の指示）

テキストから指定された除外語を取り除き、各単語の出現頻度を数える。 STOPWORDS の語を含まない、単語頻度の pandas Series を返す。指定された書き出しを使い、必要なコードをすべて含む実装を書く。

期待する動作： "Hello world Hello everyone" から hello は 2 回

配布テストは大文字と小文字をそろえた数え上げを確認する。 a ・ an ・ the ・ in ・ is ・ are だけの入力なら、結果は空になる。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/52 https://huggingface.co/datasets/bigcode/bigcodebench

例 6 ：テキストから得点と科目を取り出す

Instruct の原文（全文・コード指定を含む）

Extract data from a text and create a Pandas DataFrame. The text contains several lines, each formatted as 'Score: 85, Category: Math'. Make sure to convert the scores in integer.
The function should output with:
    DataFrame: A pandas DataFrame with extracted data.
You should write self-contained code starting with:

```
import pandas as pd
import regex as re
def task_func(text):
```

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/56 https://huggingface.co/datasets/bigcode/bigcodebench

例 6 ：日本語訳と期待する動作

日本語訳（自然言語の指示）

テキストからデータを抽出し、 pandas DataFrame を作る。各行は「 Score: 85, Category: Math 」の形式である。得点は整数に変換する。抽出したデータを含むデータフレームを返す。指定された書き出しを使い、必要なコードをすべて含む実装を書く。

期待する動作： Score 列に整数 85 、 Category 列に Math が入る

配布テストは複数行を与え、各行の得点と科目を確認する。空のテキストからは 0 行のデータフレームを返す。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/56 https://huggingface.co/datasets/bigcode/bigcodebench

例 7 ：辞書のリストを CSV と JSON に保存する

Instruct の原文（全文・コード指定を含む）

Save the list of dictionaries provided in the 'result' parameter to a CSV file (without index) and a JSON file.
The function should output with:
    None
You should write self-contained code starting with:

```
import json
import pandas as pd
def task_func(result, csv_file_path="test.csv",
json_file_path="test.json"):
```

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/60 https://huggingface.co/datasets/bigcode/bigcodebench

例 7 ：日本語訳と期待する動作

日本語訳（自然言語の指示）

result として与えた辞書のリストを、インデックスを付けない CSV と、 JSON の両方に保存する。戻り値は None とする。指定された書き出しを使い、必要なコードをすべて含む実装を書く。

期待する動作：両方のファイルに元の値を保存する

配布テストは、 hi=7 ・ bye=4 ・ from_user=11 を持つデータを保存し、CSV と JSON を読み戻して値を確認する。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/60 https://huggingface.co/datasets/bigcode/bigcodebench

例 8 ：日別売上の折れ線グラフを返す

Instruct の原文（全文・コード指定を含む）

Draw and return the daily turnover line chart from a pandas DataFrame.
The function should raise the exception for: ValueError: If 'df' is not a DataFrame or lacks 'Date' or 'Sales' columns, or has no data to plot.
The function should output with:
    Axes: Matplotlib Axes object with the line chart.
You should write self-contained code starting with:

```
import pandas as pd
import matplotlib.pyplot as plt
def task_func(df):
```

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/110 https://huggingface.co/datasets/bigcode/bigcodebench

例 8 ：日本語訳と期待する動作

日本語訳（自然言語の指示）

pandas DataFrame から日別売上の折れ線グラフを描いて返す。df が DataFrame でない場合、 Date 列または Sales 列がない場合、描くデータがない場合は ValueError を送出する。Matplotlib の Axes オブジェクトを返す。指定された書き出しを使い、必要なコードをすべて含む実装を書く。

期待する動作：タイトルは Daily Turnover 、縦軸は Sales

Axes はグラフを描く領域を表す。配布テストは描画オブジェクトの型と、折れ線上の売上値、タイトル、縦軸ラベルを調べる。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/110 https://huggingface.co/datasets/bigcode/bigcodebench

例 9 ： IP アドレスの一覧を CSV に保存する

Instruct の原文（全文・コード指定を含む）

Generates a CSV file listing all IP addresses in the specified IP range.
Each IP address is written as a row in the CSV file.
The function should output with:
    str: The path to the generated CSV file.
You should write self-contained code starting with:

```
import csv
from ipaddress import IPv4Network
def task_func(ip_range, csv_path):
```

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/145 https://huggingface.co/datasets/bigcode/bigcodebench

例 9 ：日本語訳と期待する動作

日本語訳（自然言語の指示）

指定された IP アドレスの範囲について、全アドレスの一覧を CSV に保存する。各アドレスを 1 行に書き、生成した CSV のパスを文字列として返す。指定された書き出しを使い、必要なコードをすべて含む実装を書く。

期待する動作：見出しの後に、列挙された各アドレスを書き出す

配布テストはアドレスの列挙を 2 件に置き換え、見出しを 1 回、データ行を 2 回書くことを確認する。ファイルの作成と戻り値の型も調べる。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/145 https://huggingface.co/datasets/bigcode/bigcodebench

例 10 ：カテゴリ値を整数へ符号化する

Instruct の原文（全文・コード指定を含む）

Encrypt the categorical data in a specific column of a DataFrame using LabelEncoder.
The function should output with:
    pd.DataFrame: The DataFrame with the encoded column.
You should write self-contained code starting with:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
def task_func(df: pd.DataFrame, column_name: str) ->
pd.DataFrame:
```

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/148 https://huggingface.co/datasets/bigcode/bigcodebench

例 10 ：日本語訳と期待する動作

日本語訳（自然言語の指示）

LabelEncoder を使い、 DataFrame の指定した列にあるカテゴリ値を整数に符号化する。符号化した列を含む DataFrame を返す。指定された書き出しを使い、必要なコードをすべて含む実装を書く。

期待する動作： apple ・ banana ・ cherry を 0 ・ 1 ・ 2 に対応させる

配布テストの fruit 列は apple, banana, cherry, apple, banana であり、出力は 0, 1, 2, 0, 1 になる。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/148 https://huggingface.co/datasets/bigcode/bigcodebench

例 11 ： SQL の検索結果をデータフレームにする

Instruct の原文（全文・コード指定を含む）

Query an SQLite database and return the results. This function connects to a given SQLite database, executes a given SQL query, and returns the results as a pandas DataFrame.
The function should output with:
    pd.DataFrame: A DataFrame containing the results of the executed query.
You should write self-contained code starting with:

```
import sqlite3
import pandas as pd
def task_func(db_file: str, query: str) -> pd.DataFrame:
```

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/408 https://huggingface.co/datasets/bigcode/bigcodebench

例 11 ：日本語訳と期待する動作

日本語訳（自然言語の指示）

指定された SQLite データベースに接続し、 SQL の問い合わせを実行する。検索結果を含む pandas DataFrame を返す。指定された書き出しを使い、必要なコードをすべて含む実装を書く。

期待する動作：年齢の条件と並び順を、検索結果へ反映する

配布テストは年齢の上限を条件にし、年齢の降順に並べて問い合わせる。返る全行が年齢条件と降順の両方を満たすか確認する。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/408 https://huggingface.co/datasets/bigcode/bigcodebench

例 12 ： SQLite の表を CSV に書き出す

Instruct の原文（全文・コード指定を含む）

```
Read SQLite3 table via pandas and export to a CSV file.
The function should output with:
    str: The absolute path of the exported CSV file.
You should write self-contained code starting with:
import sqlite3
import pandas as pd
import os
def task_func(db_name, table_name, csv_path="data.csv"):
```

日本語訳（自然言語の指示）

pandas を使って SQLite3 の表を読み込み、 CSV ファイルへ書き出す。書き出した CSV の絶対パスを文字列として返す。指定された書き出しを使い、必要なコードをすべて含む実装を書く。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/536 https://huggingface.co/datasets/bigcode/bigcodebench

例 12 ：テストと期待する動作

配布テストで確かめること

期待する動作： People 表の 3 行を CSV へ保存する

配布テストの People 表には Alice ・ Bob ・ Charlie がいる。保存した CSV が 3 行で Alice を含むことと、指定した出力先の絶対パスを返すことを確認する。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/536 https://huggingface.co/datasets/bigcode/bigcodebench

例 13 ：メッセージを暗号化して文字列にする

Instruct の原文（全文・コード指定を含む）

Encrypts a message with a symmetric encryption key using Fernet encryption, and then encode the encrypted message using base64.
The function should output with:
    str: The base64 encoded encrypted message. The message is first encrypted using Fernet encryption,
    then the result is base64 encoded.
You should write self-contained code starting with:

```
import base64
from cryptography.fernet import Fernet
def task_func(message, encryption_key):
```

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/26 https://huggingface.co/datasets/bigcode/bigcodebench

例 13 ：日本語訳と期待する動作

日本語訳（自然言語の指示）

対称鍵暗号の Fernet でメッセージを暗号化し、暗号化した結果をさらに base64 で符号化する。返す値は base64 で符号化した暗号文の文字列である。指定された書き出しを使い、必要なコードをすべて含む実装を書く。

期待する動作：平文とは異なる文字列を返し、不正な鍵は拒否する

配布テストは Hello, World! などを与え、文字列型で平文と異なることを確認する。長さが不正な鍵では例外が起きる。

[1] BigCode Project (n.d.; 2026-10-22 参照), BigCodeBench Dataset v0.1.4. BigCodeBench/26 https://huggingface.co/datasets/bigcode/bigcodebench

提示形式と利用条件

提示文を短くすると、使える手掛かりも変わる

項目	Complete	Instruct
主な形式	docstring	自然言語の依頼
平均の文字数	1112.5	663.2
平均の行数	33.5	11.7
解く処理・テスト	同じ	同じ

[1] Zhuo et al. (2025), BigCodeBench. ICLR 2025. https://arxiv.org/abs/2406.15877

一つの関数を、実行できる環境で評価する

- HumanEval は短く自己完結した関数の実装を中心にし、DS-1000 はデータサイエンスの処理を扱う。BigCodeBench は用途を広げ、複数のライブラリーを使う処理を問う。
- SWE-bench のようなリポジトリ全体の修正とは異なり、必要な機能を一つの関数として実装する。
- 依存ライブラリーのバージョンや実行環境はテストの動作に関わる。比較では、使った課題集合と提示形式、コードを補う処理をそろえる。

[1] Zhuo et al. (2025), BigCodeBench. ICLR 2025. https://arxiv.org/abs/2406.15877

参考文献

BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions

Terry Yue Zhuo et al. (2025). ICLR 2025.

課題の構築、ライブラリーの広がり、提示形式と採点方法、初期のモデル比較を報告する。

https://arxiv.org/abs/2406.15877

BigCodeBench Dataset

BigCode Project (n.d.). Hugging Face. Apache 2.0. 2026-10-22 参照。

1,140 課題の指示文・書き出し・参照実装・テストを、データの改訂ごとに配布する。

https://huggingface.co/datasets/bigcode/bigcodebench

参考文献

Introducing BigCodeBench-Hard

Terry Yue Zhuo (2024). Hugging Face Blog.

Stack Overflow の質問との対応と難しさに基づき、 148 課題の部分集合を選ぶ方法を説明する。

https://huggingface.co/blog/terryyz/bigcodebench-hard

BigCodeBench official repository

BigCode Project (n.d.). GitHub. Apache 2.0. 2026-10-22 参照。

コード生成、出力の整形と補完、実行テスト、環境の準備に使う実装を公開する。

https://github.com/bigcode-project/bigcodebench

参考文献

Training Agents to Self-Report Misbehavior

Bruce W. Lee, Chen Yueh-Han and Tomek Korbak (2026). arXiv.

自己申告を訓練したモデルの一般能力を調べ、 BigCodeBench を含む七つの評価で比較する。

https://arxiv.org/abs/2602.22303

BigCodeBench Leaderboard

BigCode Project (n.d.). 2026-10-22 参照。

Full と Hard 、 Complete と Instruct を切り替えて公開モデルの成績を比較する。

https://bigcode-bench.github.io/